我想用Java寫一個骰子模擬器，要有按鈕、顯示點數，還要記錄擲骰次數跟平均值，應該怎麼開始？

可以使用Java Swing來製作骰子模擬器。我建議先把程式分成「視窗介面」和「擲骰功能」兩個部分，這樣比較容易理解，也比較方便除錯。

第一步：先建立視窗介面

需要用到三個Java Swing元件：

- `JFrame`：建立400×320的視窗。
- `JLabel`：顯示骰子點數和統計資訊。
- `JButton`：建立「擲骰子」按鈕。

可以先寫出基本介面：

Java

```java
import javax.swing.*;
import java.awt.*;

public class DiceSimulator {
    public static void main(String[] args) {
        JFrame frame = new JFrame("骰子模擬器");
        frame.setSize(400, 320);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);

        JLabel diceLabel = new JLabel("0", SwingConstants.CENTER);
        diceLabel.setFont(new Font("Arial", Font.BOLD, 60));

        JButton button = new JButton("擲骰子");

        frame.add(diceLabel, BorderLayout.CENTER);
        frame.add(button, BorderLayout.SOUTH);

        frame.setVisible(true);
    }
}
```

這樣就有一個簡單的視窗，中間顯示點數，下方有按鈕。

第二步：加入隨機擲骰功能

使用`Random`產生1～6的隨機數：

第二步：加入隨機擲骰功能

使用 `Random` 產生1～6的隨機數：

```java
Random random = new Random();
int points = random.nextInt(6) + 1;
```

其中 `nextInt(6)` 會產生0～5，所以需要加1，才能模擬真正骰子的1～6點。

接著使用 `addActionListener()`，讓使用者按下按鈕時更新 `JLabel` 的文字。

第三步：加入統計功能

建立兩個變數：

```java
int count = 0;  // 擲骰次數
int total = 0;  // 點數總和
```

每次擲骰後，次數增加1，並將這次的點數加入總和。

平均值的計算方式：

```java
double average = (double) total / count;
```

使用 `String.format("%.2f", average)` 就能顯示到小數點後兩位。記得尚未擲骰時，先將平均值設為0，避免除以0。

最後再加入顏色判斷： 擲出1顯示紅色、6顯示綠色，其餘顯示黑色。

這樣就能逐步完成教授要求的功能。我的建議是先確認視窗和按鈕能正常顯示，再依序加入隨機數、統計和顏色變化，不要一開始就把所有功能混在一起。

那我要怎麼讓每次按按鈕都隨機出現1到6，而且點數是1跟6時會變不同顏色？

你可以使用Java的`Random`產生1～6的隨機數，再搭配`if-else`判斷點數，讓文字顯示不同顏色。

第一步：產生1～6的隨機點數

先在程式最上方加入：

```java
import java.util.Random;
```

接著使用：

```java
Random random = new Random();
int points = random.nextInt(6) + 1;
```

因為`nextInt(6)`只會產生0～5，所以要加1，才能得到1～6的點數。

第二步：根據點數改變顏色

使用`if-else`判斷：

```java
if (points == 1) {
    diceLabel.setForeground(Color.RED);
} else if (points == 6) {
    diceLabel.setForeground(Color.GREEN);
} else {
    diceLabel.setForeground(Color.BLACK);
}
```

這樣就能讓：

- 擲出1：顯示紅色
- 擲出6：顯示綠色
- 擲出2～5：顯示黑色

第三步：讓按鈕控制擲骰

將隨機數和顏色判斷放進`addActionListener()`，每次按下按鈕就會重新執行。

第三步：讓按鈕控制擲骰

將隨機數和顏色判斷放進 `addActionListener()`，每次按下按鈕就會重新執行。

```java
Random random = new Random();

button.addActionListener(e -> {
    int points = random.nextInt(6) + 1;

    // 更新點數
    diceLabel.setText(String.valueOf(points));

    // 更新文字顏色
    if (points == 1) {
        diceLabel.setForeground(Color.RED);
    } else if (points == 6) {
        diceLabel.setForeground(Color.GREEN);
    } else {
        diceLabel.setForeground(Color.BLACK);
    }
});
```

記得在程式上方也加入：

```java
import java.awt.Color;
```

這段程式的重點是 `addActionListener()` 會監聽按鈕的點擊事件，每按一次就產生新的隨機點數，再依照點數修改 `JLabel` 的文字和顏色。

接下來只需要在同一個按鈕事件中加入**擲骰次數、點數總和與平均值的計算**，就能完成教授要求的主要功能。

那平均值要怎麼算？教授要求顯示到小數點後兩位。

平均值的計算其實很簡單，就是**「點數總和÷擲骰次數」**，再使用 `String.format()` 讓結果顯示到小數點後兩位。

第一步：建立統計變數

先在類別中宣告兩個變數：

先在類別中宣告兩個變數：

```java
int count = 0;  // 擲骰次數
int total = 0;  // 點數總和
```

第二步：每次擲骰後更新統計

把以下程式加入按鈕的 `addActionListener()` 裡面：

```java
count++;          // 次數加1
total += points;  // 累加這次的點數

double average = (double) total / count;
```

這裡要特別注意 `(double)`，因為如果直接用兩個 `int` 相除，Java會進行整數除法，小數部分就會被捨去。

第三步：顯示小數點後兩位

假設你已經建立了上方統計用的 `statsLabel`：

```java
statsLabel.setText(String.format(
    "已擲 %d 次，總和 %d，平均 %.2f",
    count, total, average
));
```

其中 `%.2f` 就是讓平均值固定顯示到小數點後兩位，例如 `3.50`。

實際計算範例

擲骰次數	擲出的點數	總和	平均值
1	4	4	4.00
2	6	10	5.00
3	3	13	4.33
4	1	14	3.50

第四步：整合進按鈕事件

擲骰次數	擲出的點數	總和	平均值
1	4	4	4.00
2	6	10	5.00
3	3	13	4.33
4	1	14	3.50

第四步：整合進按鈕事件

Java

```java
button.addActionListener(e -> {
    int points = random.nextInt(6) + 1;

    diceLabel.setText(String.valueOf(points));

    // 判斷點數顏色
    if (points == 1) {
        diceLabel.setForeground(Color.RED);
    } else if (points == 6) {
        diceLabel.setForeground(Color.GREEN);
    } else {
        diceLabel.setForeground(Color.BLACK);
    }

    // 更新統計
    count++;
    total += points;

    double average = (double) total / count;

    statsLabel.setText(String.format(
        "已擲 %d 次，總和 %d，平均 %.2f"
```

注意： 這段假設 `count` 和 `total` 是宣告在類別中的欄位，而不是方法內的一般區域變數，這樣才能在事件處理中持續更新。

另外，程式剛啟動、還沒有擲骰時，可以先顯示：

`已擲 0 次，總和 0，平均 0.00`

這樣就不會發生除以0的問題。

完成這個部分後，你的骰子模擬器就已經具備教授要求的隨機擲骰、顏色變化，以及次數、總和和平均值統計功能了。